Buffer Overflow

Ví dụ 1:
Ví dụ 2:
Ví dụ 3:
Ví dụ 4:
Yêu cầu
- Lưu, biên dịch và chạy ví dụ 1, 2, 3
- Debug ví dụ 3 bằng gdb
- Biên dịch ví dụ 4
  - 1. Phân Tích Source Code
  - 2. Phân Tích Động (GDB Analysis)
  - 💥 Quá trình ghi đè bộ nhớ

Ví dụ 1:

```
#include <stdio.h>
#include <string.h>

int main (){
    int array[5] = {1, 2, 3, 4, 5};
    printf("%d\n", array[5] );
}
```

Ví dụ 2:

```
#include <stdio.h>
#include <string.h>

int main (){
    int array[5];
    int i;
    for (i = 0; i <= 255; i++ ){
```

```
        array[i] = 10;
    }
}
```

Ví dụ 3:

```
#include <stdio.h>
#include <string.h>

void function(int a, int b){
    int array[5];
}

main(){
    function(1,2);
    printf("This is where the return address points");
}
```

Ví dụ 4:

```
#include <stdio.h>
#include <string.h>

void return_input (void){
    char array[30];
    gets (array);
    printf("%s\n", array);
}

main(){
    return_input();
```

```
    return 0;
}
```

Yêu cầu

Lưu, biên dịch và chạy ví dụ 1, 2, 3

Với mỗi chương trình ví dụ trên **lưu thành file .c** và **biên dịch** trong máy ubuntu và **chạy**

giả sử **lưu vidu1.c**

```
sudo apt install openssh-server
```

Install GCC

- https://documentation.ubuntu.com/ubuntu-for-developers/howto/gcc-setup/

Biên dịch

```
gcc vidu1.c -o vidu1
./vidu1

gcc vidu2.c -o vidu2
./vidu2
```

Đọc hiểu chương trình và nhận xét

Debug ví dụ 3 bằng gdb

Sử dụng công cụ debug gdb

giả sử lưu chương trình ví dụ 3 thành vidu3.c

biên dịch:

```
gcc -fno-pie -no-pie -ggdb vidu3.c -o vidu3
./vidu3
```

```
gdb ./vidu3
disas main
disas function
```

Khảo sát địa chỉ lưu trữ và mã chương trình

Biên dịch ví dụ 4

Tiếp tục biên dịch ví dụ 4 để tiến hành khảo sát nội dung stack khi chạy chương trình

Ví dụ lưu chương trình ví dụ 4 thành vidu4.c

1. Phân Tích Source Code

```
// vidu4.c
#include <stdio.h>
#include <string.h>

void return_input (void){
    char array[30];      // Buffer được cấp phát 30 bytes
    gets (array);        // ❌ LỖI: gets() không kiểm soát độ
dài đầu vào
    printf("%s\n", array);
}

main(){
    return_input();      // Gọi hàm gây lỗi
    return 0;
}
```

- **Vulnerability:** Hàm `gets(array)` sẽ đọc dữ liệu từ bàn phím cho đến khi gặp ký tự xuống dòng (`\n`) mà không quan tâm `array` chỉ chứa được 30 ký tự.
- **Attack Vector:** Nhập một chuỗi dài hơn kích thước stack frame của `return_input` để ghi đè lên các giá trị quan trọng (EBP, Return Address).

2. Phân Tích Động (GDB Analysis)

▼ **Script khai thác**

```
cat vidu4.c

#include <stdio.h>
#include <string.h>

void return_input (void){
    char array[30];
    gets (array);
    printf("%s\n", array);
}

main(){
    return_input();
    return 0;
}

gcc -fno-pie -no-pie -mpreferred-stack-boundary=4 -fno-sta
ck-protector -ggdb vidu4.c -o vidu4

./vidu4

gdb ./vidu4
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.5) 7.11.1
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.or
g/licenses/gpl.html>
This is free software: you are free to change and redistri
bute it.
There is NO WARRANTY, to the extent permitted by law.  Typ
e "show copying"
and "show warranty" for details.
This GDB was configured as "i686-linux-gnu".
```

```
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources onli
ne at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "wor
d"...
Reading symbols from ./vidu4...done.

(gdb) disas main
Dump of assembler code for function main:
   0x08048462 <+0>:     lea    0x4(%esp),%ecx
   0x08048466 <+4>:     and    $0xfffffff0,%esp
   0x08048469 <+7>:     pushl  -0x4(%ecx)
   0x0804846c <+10>:    push   %ebp
   0x0804846d <+11>:    mov    %esp,%ebp
   0x0804846f <+13>:    push   %ecx
   0x08048470 <+14>:    sub    $0x4,%esp
   0x08048473 <+17>:    call   0x804843b <return_input>
   0x08048478 <+22>:    mov    $0x0,%eax
   0x0804847d <+27>:    add    $0x4,%esp
   0x08048480 <+30>:    pop    %ecx
   0x08048481 <+31>:    pop    %ebp
   0x08048482 <+32>:    lea    -0x4(%ecx),%esp
   0x08048485 <+35>:    ret
End of assembler dump.

(gdb) disas return_input
Dump of assembler code for function return_input:
   0x0804843b <+0>:     push   %ebp
   0x0804843c <+1>:     mov    %esp,%ebp
   0x0804843e <+3>:     sub    $0x28,%esp
   0x08048441 <+6>:     sub    $0xc,%esp
   0x08048444 <+9>:     lea    -0x26(%ebp),%eax
```

```
   0x08048447 <+12>:    push   %eax
   0x08048448 <+13>:    call   0x8048300 <gets@plt>
   0x0804844d <+18>:    add    $0x10,%esp
   0x08048450 <+21>:    sub    $0xc,%esp
   0x08048453 <+24>:    lea    -0x26(%ebp),%eax
   0x08048456 <+27>:    push   %eax
   0x08048457 <+28>:    call   0x8048310 <puts@plt>
   0x0804845c <+33>:    add    $0x10,%esp
   0x0804845f <+36>:    nop
   0x08048460 <+37>:    leave
   0x08048461 <+38>:    ret
End of assembler dump.

(gdb) b* 0x08048448
Breakpoint 1 at 0x8048448: file vidu4.c, line 6.

(gdb) b* 0x08048457
Breakpoint 2 at 0x8048457: file vidu4.c, line 7.

(gdb) run
Starting program: /home/osboxes/Desktop/lab1/vidu4

Breakpoint 1, 0x08048448 in return_input () at vidu4.c:6
6           gets (array);

(gdb) x/20x $esp
0xbffff560:     0xbffff572      0x0000002f      0xb7e15dc8
0xb7fd51b0
0xbffff570:     0x00008000      0xb7fbb000      0xb7fb9244
0xb7e210ec
0xbffff580:     0x00000001      0x00000000      0xb7e37a50
0x080484db
0xbffff590:     0x00000001      0xbffff654      0xbffff5a8
0x08048478
0xbffff5a0:     0xb7fbb3dc      0xbffff5c0      0x00000000
0xb7e21637
```

```
(gdb) ni
AAAAAAAAAAAAAAAAAAAABBBBBBBBBBBBBBBBBBBBCCCCCCCCCC
0x0804844d      6           gets (array);

(gdb) x/20x 0xbffff560
0xbffff560:     0xbffff572      0x0000002f      0xb7e15dc8
0xb7fd51b0
0xbffff570:     0x41418000      0x41414141      0x41414141
0x41414141
0xbffff580:     0x41414141      0x42424141      0x42424242
0x42424242
0xbffff590:     0x42424242      0x42424242      0x43434242
0x43434343
0xbffff5a0:     0x43434343      0xbffff500      0x00000000
0xb7e21637

(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/osboxes/Desktop/lab1/vidu4

Breakpoint 1, 0x08048448 in return_input () at vidu4.c:6
6           gets (array);

(gdb) ni
AAAAAAAAAAAAAAAAAAAABBBBBBBBBBBBBBBBBBBBCCCCCCCCCC
0x0804844d      6           gets (array);

(gdb) disas main
Dump of assembler code for function main:
   0x08048462 <+0>:     lea    0x4(%esp),%ecx
   0x08048466 <+4>:     and    $0xfffffff0,%esp
   0x08048469 <+7>:     pushl  -0x4(%ecx)
   0x0804846c <+10>:    push   %ebp
   0x0804846d <+11>:    mov    %esp,%ebp
```

```
   0x0804846f <+13>:    push   %ecx
   0x08048470 <+14>:    sub    $0x4,%esp
   0x08048473 <+17>:    call   0x804843b <return_input>
   0x08048478 <+22>:    mov    $0x0,%eax
   0x0804847d <+27>:    add    $0x4,%esp
   0x08048480 <+30>:    pop    %ecx
   0x08048481 <+31>:    pop    %ebp
   0x08048482 <+32>:    lea    -0x4(%ecx),%esp
   0x08048485 <+35>:    ret
End of assembler dump.

(gdb) disas return_input
Dump of assembler code for function return_input:
   0x0804843b <+0>:     push   %ebp
   0x0804843c <+1>:     mov    %esp,%ebp
   0x0804843e <+3>:     sub    $0x28,%esp
   0x08048441 <+6>:     sub    $0xc,%esp
   0x08048444 <+9>:     lea    -0x26(%ebp),%eax
   0x08048447 <+12>:    push   %eax
   0x08048448 <+13>:    call   0x8048300 <gets@plt>
=> 0x0804844d <+18>:    add    $0x10,%esp
   0x08048450 <+21>:    sub    $0xc,%esp
   0x08048453 <+24>:    lea    -0x26(%ebp),%eax
   0x08048456 <+27>:    push   %eax
   0x08048457 <+28>:    call   0x8048310 <puts@plt>
   0x0804845c <+33>:    add    $0x10,%esp
   0x0804845f <+36>:    nop
   0x08048460 <+37>:    leave
   0x08048461 <+38>:    ret
End of assembler dump.
(gdb)
```

Bước 1: Xác định vị trí Buffer trong Stack

Dựa vào mã Assembly của hàm `return_input`:

```
(gdb) disas return_input
...
0x0804843b <+0>:    push   %ebp
0x0804843c <+1>:    mov    %esp,%ebp        ; Thiết lập đáy S
tack Frame (EBP)
...
0x08048444 <+9>:    lea    -0x26(%ebp),%eax ; Load địa chỉ Bu
ffer vào EAX
...
```

> - Lệnh `lea -0x26(%ebp), %eax` cho biết biến `array` bắt đầu tại địa chỉ `EBP - 0x26`.
> - `0x26` (hex) = `38` (decimal).
> - **Kết luận 1:** Buffer nằm cách đáy stack (Saved EBP) đúng **38 bytes**.

Bước 2: Xác định các địa chỉ quan trọng (Trước khi nhập)

Tại breakpoint `gets(array)`, ta xem xét bộ nhớ Stack:

```
(gdb) x/20x $esp
...
0xbffff590:     0x00000001  0xbffff654  0xbffff5a8  0x0804847
8
```

Dòng `0xbffff590` chứa các thông tin quan trọng nhất để điều khiển luồng chương trình. Ta phân tích từng ô nhớ (mỗi ô 4 bytes):

Offset (Hex)	Địa chỉ	Giá trị	Ý nghĩa
`+0x0`	`0xbffff590`	`0x00000001`	Biến cục bộ / Padding
`+0x4`	`0xbffff594`	`0xbffff654`	Biến cục bộ / Padding
`+0x8`	**`0xbffff598`**	**`0xbffff5a8`**	**Saved EBP** (EBP của hàm main)
`+0xC`	**`0xbffff59c`**	**`0x08048478`**	**Return Address** (Địa chỉ quay về main)

📍 Bước 3: Tính toán địa chỉ bắt đầu Buffer

Ta đã biết Buffer nằm tại `EBP - 0x26`.

- Saved EBP nằm tại: `0xbffff598` (Lấy từ bảng trên).
- Buffer Start = `0xbffff598` - `0x26` = **`0xbffff572`**.

3. Kịch Bản Tấn Công

Nhập payload `AAAAAAAAAAAAAAAAAAAABBBBBBBBBBBBBBBBBBBBCCCCCCCCCC`

- **20 ký tự 'A'** (`0x41`)
- **20 ký tự 'B'** (`0x42`)
- **10 ký tự 'C'** (`0x43`)
- **Tổng cộng:** 50 bytes.

💥 Quá trình ghi đè bộ nhớ

Sơ đồ chi tiết cách 50 bytes này lấp đầy Stack từ thấp (`572`) lên cao (`59c`).

Vùng nhớ	Địa chỉ bắt đầu	Độ rộng (Bytes)	Dữ liệu Input	Trạng thái
Buffer (Phần khai báo)	`0xbffff572`	30	20 'A' + 10 'B'	✅ Hợp lệ
Padding (Vùng đệm của Compiler)	`0xbffff590`	8	8 'B'	⚠️ Tràn nhẹ (Không nguy hiểm)
Saved EBP (Con trỏ Frame cũ)	`0xbffff598`	4	2 'B' + 2 'C'	🚨 **Saved EBP bị hỏng**
Return Address (Hướng đi tiếp theo)	`0xbffff59c`	4	4 'C' (`0x43434343`)	💀 **MẤT QUYỀN ĐIỀU KHIỂN**
Dư thừa	`0xbffff5a0`	4	4 'C'	Ghi tiếp lên trên...

Kết luận

Khi hàm `return_input` thực hiện lệnh `ret` (return):

1. CPU sẽ nhảy (POP EIP) tới địa chỉ nằm tại `0xbffff59c`.
2. Địa chỉ đó hiện tại là `0x43434343`.
3. Địa chỉ `0x43434343` không phải là vùng nhớ hợp lệ chứa mã lệnh.
4. Chương trình báo lỗi **Segmentation Fault** (Segfault).